




UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO

# Framework para Sistemas de Gerenciamento de Aluguel de Apartamentos e Casas Usando Microservices e Componentes de Software [e Agentes de IA]

Disciplina: Reuso de Software e Metodologias Ágeis

Francys Samuel Oliveira Pereira dos Santos

2026

## Sumário

1 Enunciado . . . . . 3
2 Metodologia Ágil . . . . . 3
2.1 Sprint 1 — Infraestrutura Base e Modelagem 3
2.2 Sprint 2 — Framework, Hotspots e Componentes 4
2.3 Sprint 3 — Aplicações, Microsserviços e Interface 4
3 Técnicas de Reuso de Software Utilizadas . . . . . 4
3.1 Framework com Padrão Template Method . . . . . 4
3.2 Componentes de Software Reutilizáveis . . . . . 4
3.3 Arquitetura de Microsserviços . . . . . 5
3.4 Agentes de IA com Ciclo ReAct . . . . . 5
4 Descrição do Desenvolvimento . . . . . 5
4.1 Framework RentalFramework . . . . . 5
4.2 Hotspots e Frozen Spots . . . . . 5
4.3 Aplicações Construídas . . . . . 6
4.4 Integração com o Sistema de Microsserviços 6
5 Product Backlog e Sprint Backlog . . . . . 6
5.1 Product Backlog . . . . . 6
5.2 Sprint Backlog — Sprint 1: Framework Base . 7
5.3 Sprint Backlog — Sprint 2: Componentes . . . . 7
5.4 Sprint Backlog — Sprint 3: Aplicações e Microsserviços 7
6 Arquitetura do Sistema . . . . . 8
6.1 Diagrama de Classes do Framework . . . . . 8
6.2 Código do Template Method . . . . . 9
6.3 Código das Interfaces (Hotspots) . . . . . 11
6.4 Código das 2 Aplicações . . . . . 12
6.5 Como Criar uma Nova Aplicação com o Framework 13
6.6 Arquitetura de Microsserviços . . . . . 14
6.7 Especificação da API Gateway . . . . . 15
6.8 Especificação dos Microsserviços . . . . . 16
6.9 Microsserviços em Execução e Portas Associadas 16
7 Tecnologias Utilizadas . . . . . 16
7.1 Framework e Aplicações . . . . . 16
7.2 Microsserviços . . . . . 17
7.3 Frontend e Infraestrutura . . . . . 17
8 Funcionamento do Sistema . . . . . 17
8.1 Telas do Sistema e Funcionamento . . . . . 17
8.2 Saída das Aplicações no Terminal . . . . . 18
8.3 Ciclo de Raciocínio do Agente de IA . . . . . 18
9 GitHub . . . . . 19
10 Referências Bibliográficas . . . . . 19

# 1 Enunciado

Este relatório descreve o desenvolvimento de um Framework de Gerenciamento de Aluguel de Apartamentos e Casas utilizando o padrão de projeto comportamental Template Method, integrado a uma arquitetura de microsserviços e potencializado por Agentes de Inteligência Artificial (AI Agents). O projeto foi desenvolvido para a disciplina de Reuso de Software e Metodologias Ágeis, lecionada pelo professor Dr. Arturo Hernández Domínguez, no Instituto de Computação da Universidade Federal de Alagoas (UFAL).

O objetivo principal consistiu em construir um framework reutilizável de software para a criação e especialização de aplicações imobiliárias, isolando o núcleo invariante do processo de locação (frozen spots) e fornecendo pontos de adaptação (hotspots) para suportar diferentes modelos de negócio, tais como aluguéis residenciais familiares tradicionais e aluguéis estudantis/temporada flexíveis. O sistema contempla:

• Framework orientado a objetos com padrão Template Method e 4 hotspots;

• 8 componentes de software reutilizáveis (Precificação, Análise de Risco, Agentes de IA e Armazenamento);

• 2 aplicações completas construídas reusando o framework: AluguelResidencialApp e AluguelUniversitarioApp;

• Arquitetura distribuída com 5 microsserviços autônomos operando em portas de rede dedicadas (8000 a 8004);

• Agente de Inteligência Artificial ReAct com transparência no ciclo de percepção, raciocínio e ação;

• Interface gráfica interativa construída em Streamlit para demonstração das camadas.

O código-fonte completo está disponível em: https://github.com/samueeloliveiraa/framework-aluguel-rsma

# 2 Metodologia Ágil

A metodologia ágil adotada no desenvolvimento do projeto foi o Scrum, caracterizada por entregas incrementais organizadas em ciclos curtos denominados Sprints. O Scrum permitiu adaptações contínuas na modelagem dos contratos de interface dos componentes e na orquestração dos microsserviços. O ciclo foi dividido em 3 sprints principais:

## 2.1 Sprint 1 — Infraestrutura Base e Modelagem

• Levantamento das regras de negócio de locação imobiliária (apartamentos e casas);
• Identificação e separação formal dos frozen spots e dos hotspots do framework;
• Implementação da classe base abstrata RentalFramework com módulo abc do Python;
• Criação dos contratos de interface dos 4 hotspots: IPricingEngine, IRiskEvaluator, IAIAgentEngine e IStorageStrategy;
• Mapeamento da topologia de portas de rede para a arquitetura de microsserviços.

### 2.2 Sprint 2 — Framework, Hotspots e Componentes

• Desenvolvimento dos 8 componentes reutilizáveis implementando as interfaces dos hotspots;
• Implementação dos motores de precificação: CStandardPricingEngine e CSeasonalPricingEngine;
• Implementação dos avaliadores de crédito: CConservativeRiskEvaluator e CFlexibleRiskEvaluator;
• Implementação dos motores de inteligência: CReActAIAgent e CRuleBasedAIAgent;
• Implementação das estratégias de armazenamento: CSQLiteStorage e CMemoryStorage;
• Construção das duas aplicações derivadas que reusam o framework: AluguelResidencialApp e AluguelUniversitarioApp.

### 2.3 Sprint 3 — Aplicações, Microsserviços e Interface

• Implementação do API Gateway centralizado (Porta 8000) com proxy reverso e agregação de health checks;
• Implementação dos 4 microsserviços REST autônomos: PropertyService (:8001), ContractService (:8002), AIService (:8003) e RiskService (:8004);
• Criação do orquestrador de microsserviços simultâneos em background (run_all.py);
• Construção da interface gráfica em Streamlit destacando as aplicações do framework, catálogo, contratos e agente ReAct;
• Elaboração da documentação técnica e compilação do relatório acadêmico.

## 3 Técnicas de Reuso de Software Utilizadas

As principais técnicas de reuso de software aplicadas neste projeto foram:

### 3.1 Framework com Padrão Template Method

O RentalFramework foi construído segundo o padrão comportamental Template Method (Gamma et al., 1994). A classe abstrata define o esqueleto canônico do algoritmo de atendimento de locação no método fixo processar_fluxo_locacao() (frozen spot). Este método invoca os 4 hotspots na ordem correta: 1. configurar_pricing() — define a precificação e taxas de entrada;
2. configurar_risco() — define as regras de crédito e comprometimento de renda;
3. configurar_agente_ia() — define o motor de raciocínio inteligente;
4. configurar_storage() — define a estratégia de persistência.

### 3.2 Componentes de Software Reutilizáveis

Foram desenvolvidos 8 componentes de software modulares e independentes, com interfaces estritas e alta coesão, permitindo sua reutilização direta em qualquer aplicação do ecossistema sem necessidade de modificação interna.

### 3.3 Arquitetura de Microsserviços

A decomposição em microsserviços promove o reuso sistêmico através da divisão em serviços independentes e autônomos. Cada microsserviço cuida de seu bounded context (Imóveis, Contratos, IA, Risco) e expõe suas funcionalidades através de endpoints HTTP REST padronizados, desacoplando completamente clientes e provedores.

### 3.4 Agentes de IA com Ciclo ReAct

O reuso de componentes inteligentes baseia-se no padrão ReAct (Reasoning + Acting), em que o agente recebe o contexto do inquilino e executa um loop auditável composto por reflexão analítica (Thought), chamada de ferramentas e microsserviços (Action) e consolidação de evidências (Observation) para emitir pareceres explicados.

## 4 Descrição do Desenvolvimento

O desenvolvimento seguiu os princípios ágeis do Scrum integrados a técnicas avançadas de reuso de software. O projeto foi estruturado em duas camadas principais: a Camada de Reuso (Framework Orientado a Objetos e Componentes) e a Camada Distribuída (Microsserviços de Rede e API Gateway).

### 4.1 Framework RentalFramework

O framework foi desenvolvido em Python 3 utilizando o módulo abc para implementar classes abstratas puras e interfaces formais. A arquitetura segue o modelo híbrido (caixa-branca e caixa-preta): permite tanto a especialização por herança da classe RentalFramework (caixa-branca) quanto a montagem composicional a partir dos componentes pré-fabricados (caixa-preta).

### 4.2 Hotspots e Frozen Spots

**Frozen Spots (Partes Imutáveis):**

• inicializar_framework(): Template Method de inicialização dos 4 hotspots;
• processar_fluxo_locacao(): Template Method de execução canônica do fluxo de atendimento;
• get_framework_info(): Metadados de versão e identificação oficial da arquitetura.

**Hotspots (Pontos de Flexibilidade e Variação):**

• configurar_pricing(): Hotspot 1 — seleção do cálculo financeiro e taxas de caução;
• configurar_risco(): Hotspot 2 — seleção da política de crédito e margem de renda;
• configurar_agente_ia(): Hotspot 3 — seleção do motor inteligente de match;
• configurar_storage(): Hotspot 4 — seleção da estratégia de persistência.

### 4.3 Aplicações Construídas

Foram desenvolvidas 2 aplicações completas especializando o RentalFramework:

**1. AluguelResidencialApp — Locação Familiar Tradicional:**
• Público: Famílias buscando apartamentos ou casas de médio e alto padrão;
• Pricing: CStandardPricingEngine (caução integral de 3 meses, taxa adm 5%, seguro incêndio 1.5%);
• Risco: CConservativeRiskEvaluator (teto rígido de 30% da renda, score mínimo 700);
• Agente IA: CReActAIAgent (raciocínio analítico ReAct com ponderação de quartos, vagas e quintal);
• Storage: CSQLiteStorage (persistência relacional em /tmp/aluguel_residencial.db).

**2. AluguelUniversitarioApp — Locação Estudantil e Temporada:**
• Público: Estudantes universitários, pós-graduandos e locações de studios;
• Pricing: CSeasonalPricingEngine (caução reduzida para 1 mês, desconto de 10% por pontualidade);
• Risco: CFlexibleRiskEvaluator (tolerância ampliada para até 45% da renda com comprovante acadêmico);
• Agente IA: CRuleBasedAIAgent (heurística determinística focada em menor custo global);
• Storage: CSQLiteStorage (persistência relacional em /tmp/aluguel_universitario.db).

### 4.4 Integração com o Sistema de Microsserviços

O framework conecta-se à malha de microsserviços via API Gateway (:8000). A interface visual em Streamlit permite que o operador teste a execução do Template Method em ambas as aplicações e visualize as chamadas refletidas instantaneamente no barramento de eventos e nos logs dos microsserviços.

## 5 Product Backlog e Sprint Backlog

### 5.1 Product Backlog

| ID | História de Usuário / Funcionalidade | Prioridade | Estimativa |
|---|---|---|---|
| 1 | Implementar RentalFramework com Template Method | Muito Alta | 16h |
| 2 | Criar interfaces dos 4 hotspots abstratos | Muito Alta | 8h |
| 3 | Desenvolver componentes de Precificação (IPricingEngine) | Alta | 6h |
| 4 | Desenvolver componentes de Avaliação de Risco (IRiskEvaluator) | Alta | 6h |
| 5 | Desenvolver componentes de Agente de IA com padrão ReAct | Alta | 10h |
| 6 | Desenvolver componentes de Armazenamento (IStorageStrategy) | Média | 6h |
| 7 | Construir aplicação AluguelResidencialApp | Alta | 6h |
| 8 | Construir aplicação AluguelUniversitarioApp | Alta | 6h |
| 9 | Implementar API Gateway com proxy reverso e health check | Alta | 8h |
| 10 | Implementar os 4 microsserviços autônomos REST | Muito Alta | 14h |
| 11 | Criar orquestrador simultâneo de microsserviços (run_all.py) | Média | 4h |
| 12 | Desenvolver interface gráfica em Streamlit | Alta | 12h |

## 5.2 Sprint Backlog — Sprint 1: Framework Base

| ID | Funcionalidade / Tarefa | Nível | Estimativa |
|---|---|---|---|
| 1.1 | Classe abstrata RentalFramework com Template Method | Alto | 8h |
| 1.2 | Interface IPricingEngine (Hotspot 1) | Alto | 2h |
| 1.3 | Interface IRiskEvaluator (Hotspot 2) | Alto | 2h |
| 1.4 | Interface IAIAgentEngine (Hotspot 3) | Alto | 3h |
| 1.5 | Interface IStorageStrategy (Hotspot 4) | Médio | 2h |
| 1.6 | Mapeamento das portas e topologia dos microsserviços | Alto | 4h |

## 5.3 Sprint Backlog — Sprint 2: Componentes de Software

| ID | Componente Implementado | Nível | Estimativa |
|---|---|---|---|
| 2.1 | CStandardPricingEngine (Caução 3x, taxas de administração) | Médio | 3h |
| 2.2 | CSeasonalPricingEngine (Caução 1x, desconto pontualidade) | Médio | 3h |
| 2.3 | CConservativeRiskEvaluator (Teto 30%, score 700+) | Médio | 3h |
| 2.4 | CFlexibleRiskEvaluator (Teto 45%, comprovação acadêmica) | Médio | 3h |
| 2.5 | CReActAIAgent (Loop ReAct e scoring multicritério) | Alto | 6h |
| 2.6 | CRuleBasedAIAgent (Heurística rápida de menor custo) | Médio | 3h |
| 2.7 | CSQLiteStorage e CMemoryStorage (Persistência relacional) | Médio | 4h |

## 5.4 Sprint Backlog — Sprint 3: Aplicações e Microsserviços

| ID | Funcionalidade / Entrega | Nível | Estimativa |
|---|---|---|---|
| 3.1 | Aplicação AluguelResidencialApp | Médio | 4h |
| 3.2 | Aplicação AluguelUniversitarioApp | Médio | 4h |
| 3.3 | 5 Microsserviços autônomos em portas reais (8000 a 8004) | Alto | 8h |
| 3.4 | API Gateway com proxy reverso e health checks unificados | Alto | 6h |
| 3.5 | Interface gráfica interativa em Streamlit (app.py) | Alto | 8h |
| 3.6 | Documentação completa e compilação do relatório acadêmico | Alto | 6h |

# 6 Arquitetura do Sistema

## 6.1 Diagrama de Classes do Framework

A Figura 1 apresenta o Diagrama de Classes do RentalFramework, seguindo o padrão da literatura de engenharia de software (Gamma et al., 1994) e a modelagem de referência da disciplina (Framework para Pagamentos na Web, UFAL 2025):

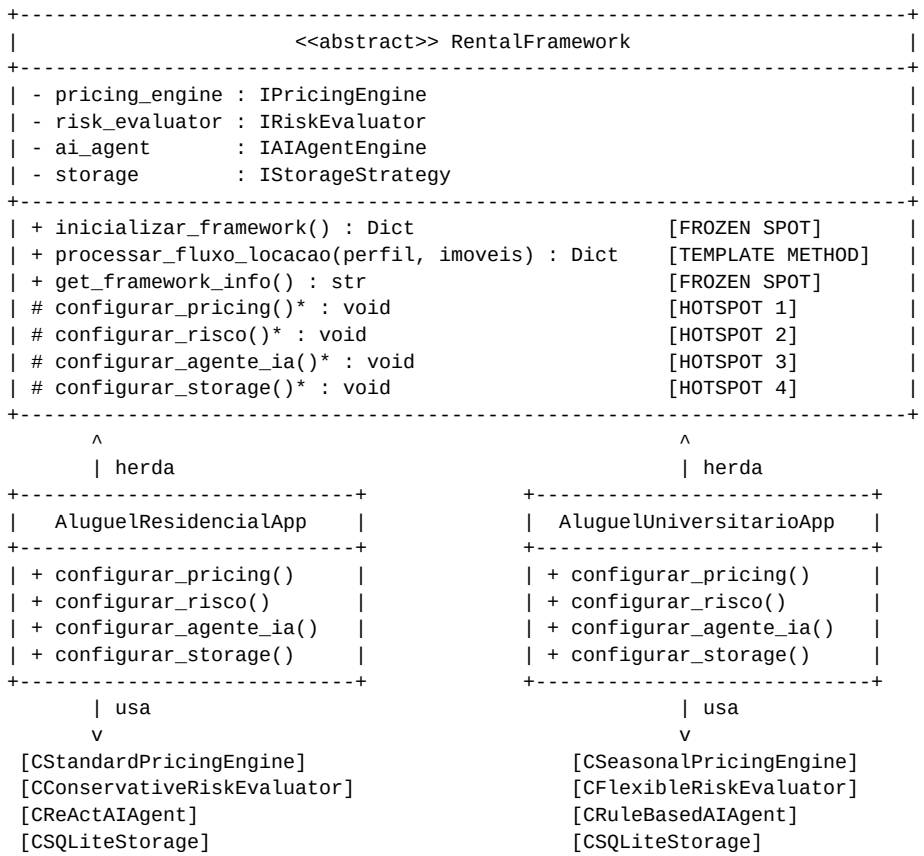



Figura 1: Diagrama de Classes do RentalFramework com Template Method, Hotspots e Componentes

## 6.2 Código do Template Method

O código abaixo apresenta a classe abstrata RentalFramework contendo os frozen spots e a declaração dos hotspots:

```
# Listing 1: Classe Abstrata RentalFramework (Núcleo do Framework)
from abc import ABC, abstractmethod
from typing import Dict, Any, List
from framework_aluguel.framework.interfaces.i_pricing_engine import IPricingEngine
from framework_aluguel.framework.interfaces.i_risk_evaluator import IRiskEvaluator
from framework_aluguel.framework.interfaces.i_ai_agent import IAIAgentEngine
from framework_aluguel.framework.interfaces.i_storage_strategy import IStorageStrategy

class RentalFramework(ABC):
    """Classe base abstrata com padrão Template Method para locação de imóveis."""
    def __init__(self):
        self.pricing_engine: IPricingEngine = None
        self.risk_evaluator: IRiskEvaluator = None
        self.ai_agent: IAIAgentEngine = None
        self.storage: IStorageStrategy = None

    # FROZEN SPOT 1: Template Method de Inicialização
    def inicializar_framework(self) -> Dict[str, Any]:
        self.configurar_pricing()
        self.configurar_risco()
        self.configurar_agente_ia()
        self.configurar_storage()
        return self.obter_configuracao_consolidada()

    # FROZEN SPOT 2: Template Method de Execução do Atendimento Canônico
    def processar_fluxo_locacao(self, perfil_inquilino: Dict[str, Any], catalogo_imoveis: List[Dict[str, Any]]) -> Di
        if not all([self.pricing_engine, self.risk_evaluator, self.ai_agent, self.storage]):
            self.inicializar_framework()

        # Etapa 1: Enriquecimento financeiro via Hotspot 1
        imoveis_precificados = []
        for im in catalogo_imoveis:
            item = dict(im)
            item["custos"] = self.pricing_engine.calcular_custos(
                aluguel_base=im.get("aluguel_base", 0.0),
                condominio=im.get("condominio", 0.0),
                iptu=im.get("iptu", 0.0)
            )
            imoveis_precificados.append(item)

        # Etapa 2: Inteligência e matching pelo Agente de IA via Hotspots 2 e 3
        resultado_agente = self.ai_agent.raciocinar_e_recomendar(
            perfil_inquilino=perfil_inquilino,
            catalogo_imoveis=imoveis_precificados
        )

        return {
            "framework": self.get_framework_info(),
            "perfil_inquilino": perfil_inquilino,
            "componentes_ativos": self.obter_configuracao_consolidada(),
            "resultado_agente": resultado_agente
        }

    # HOTSPOTS: Métodos abstratos implementados obrigatoriamente pelas subclasses
    @abstractmethod
    def configurar_pricing(self) -> None: pass
    @abstractmethod
    def configurar_risco(self) -> None: pass
    @abstractmethod
    def configurar_agente_ia(self) -> None: pass
    @abstractmethod
    def configurar_storage(self) -> None: pass

    # FROZEN SPOT 3: Metadados imutáveis
```

```
    def get_framework_info(self) -> str:
        return "RentalFramework v1.0 - UFAL IC 2026"
```

### 6.3 Código das Interfaces (Hotspots)

Abaixo estão as 4 interfaces formais que definem os contratos dos pontos de adaptação:

```
# Listing 2: Interface IPricingEngine (Hotspot 1)
from abc import ABC, abstractmethod
from typing import Dict, Any

class IPricingEngine(ABC):
    @abstractmethod
    def calcular_custos(self, aluguel_base: float, condominio: float, iptu: float) -> Dict[str, Any]: pass
    @abstractmethod
    def get_engine_name(self) -> str: pass

# Listing 3: Interface IRiskEvaluator (Hotspot 2)
class IRiskEvaluator(ABC):
    @abstractmethod
    def avaliar_credito(self, renda_mensal: float, custo_total_mensal: float, score_credito: int = 700) -> Dict[str,
    @abstractmethod
    def get_evaluator_name(self) -> str: pass

# Listing 4: Interface IAIAgentEngine (Hotspot 3)
class IAIAgentEngine(ABC):
    @abstractmethod
    def raciocinar_e_recomendar(self, perfil_inquilino: Dict[str, Any], catalogo_imoveis: List[Dict[str, Any]]) -> Di
    @abstractmethod
    def get_agent_name(self) -> str: pass

# Listing 5: Interface IStorageStrategy (Hotspot 4)
class IStorageStrategy(ABC):
    @abstractmethod
    def salvar_contrato(self, contrato: Dict[str, Any]) -> bool: pass
    @abstractmethod
    def listar_contratos(self) -> List[Dict[str, Any]]: pass
    @abstractmethod
    def get_storage_name(self) -> str: pass
```

## 6.4 Código das 2 Aplicações

As listagens a seguir apresentam o código completo e documentado das duas aplicações derivadas:

```
# Listing 6: Aplicação 1 - AluguelResidencialApp (Residencial Tradicional)
from framework_aluguel.framework.rental_framework import RentalFramework
from framework_aluguel.framework.components.pricing.c_standard_pricing import CStandardPricingEngine
from framework_aluguel.framework.components.risk.c_conservative_risk import CConservativeRiskEvaluator
from framework_aluguel.framework.components.ai.c_react_agent import CReActAIAgent
from framework_aluguel.framework.components.storage.c_sqlite_storage import CSQLiteStorage

class AluguelResidencialApp(RentalFramework):
    """Aplicação para locação familiar tradicional de casas e apartamentos."""
    def configurar_pricing(self) -> None:
        self.pricing_engine = CStandardPricingEngine(taxa_adm=0.05, meses_caucao=3)

    def configurar_risco(self) -> None:
        self.risk_evaluator = CConservativeRiskEvaluator()

    def configurar_agente_ia(self) -> None:
        self.ai_agent = CReActAIAgent()

    def configurar_storage(self) -> None:
        self.storage = CSQLiteStorage(db_path="/tmp/aluguel_residencial.db")

# Listing 7: Aplicação 2 - AluguelUniversitarioApp (Estudantil / Temporada)
from framework_aluguel.framework.components.pricing.c_seasonal_pricing import CSeasonalPricingEngine
from framework_aluguel.framework.components.risk.c_flexible_risk import CFlexibleRiskEvaluator
from framework_aluguel.framework.components.ai.c_rule_agent import CRuleBasedAIAgent

class AluguelUniversitarioApp(RentalFramework):
    """Aplicação para locação estudantil e studios universitários."""
    def configurar_pricing(self) -> None:
        self.pricing_engine = CSeasonalPricingEngine(desconto_pontualidade_pct=0.10)

    def configurar_risco(self) -> None:
        self.risk_evaluator = CFlexibleRiskEvaluator()

    def configurar_agente_ia(self) -> None:
        self.ai_agent = CRuleBasedAIAgent()

    def configurar_storage(self) -> None:
        self.storage = CSQLiteStorage(db_path="/tmp/aluguel_universitario.db")
```

### 6.5 Como Criar uma Nova Aplicação com o Framework

Para criar uma nova aplicação reusando o RentalFramework, o desenvolvedor executa 3 passos simples:

1. Herdar a classe RentalFramework;
2. Implementar os 4 métodos abstratos associando os componentes desejados aos hotspots;
3. Invocar inicializar_framework() ou processar_fluxo_locacao().

O Listing 8 demonstra a criação de uma terceira aplicação especializada em curtas estadias:

```
# Listing 8: Criação de Nova Aplicação - AluguelTemporadaApp (Short-Stay)
from framework_aluguel.framework.rental_framework import RentalFramework
from framework_aluguel.framework.components.pricing.c_seasonal_pricing import CSeasonalPricingEngine
from framework_aluguel.framework.components.risk.c_flexible_risk import CFlexibleRiskEvaluator
from framework_aluguel.framework.components.ai.c_rule_agent import CRuleBasedAIAgent
from framework_aluguel.framework.components.storage.c_memory_storage import CMemoryStorage

class AluguelTemporadaApp(RentalFramework):
    """Aplicação 3: Aluguel por temporada e curtas estadias (Airbnb style)."""
    def configurar_pricing(self) -> None:
        # Hotspot 1: 5% de desconto de pontualidade
        self.pricing_engine = CSeasonalPricingEngine(desconto_pontualidade_pct=0.05)

    def configurar_risco(self) -> None:
        # Hotspot 2: Política de crédito flexível
        self.risk_evaluator = CFlexibleRiskEvaluator()

    def configurar_agente_ia(self) -> None:
        # Hotspot 3: Motor determinístico ágil de menor custo
        self.ai_agent = CRuleBasedAIAgent()

    def configurar_storage(self) -> None:
        # Hotspot 4: Persistência volátil em memória para estadias curtas
        self.storage = CMemoryStorage()

if __name__ == "__main__":
    app = AluguelTemporadaApp()
    info = app.inicializar_framework()
    print("Nova Aplicação Temporada pronta para uso:", info)
```

### 6.6 Arquitetura de Microsserviços

A Figura 2 apresenta a arquitetura distribuída composta por 5 microsserviços autônomos:

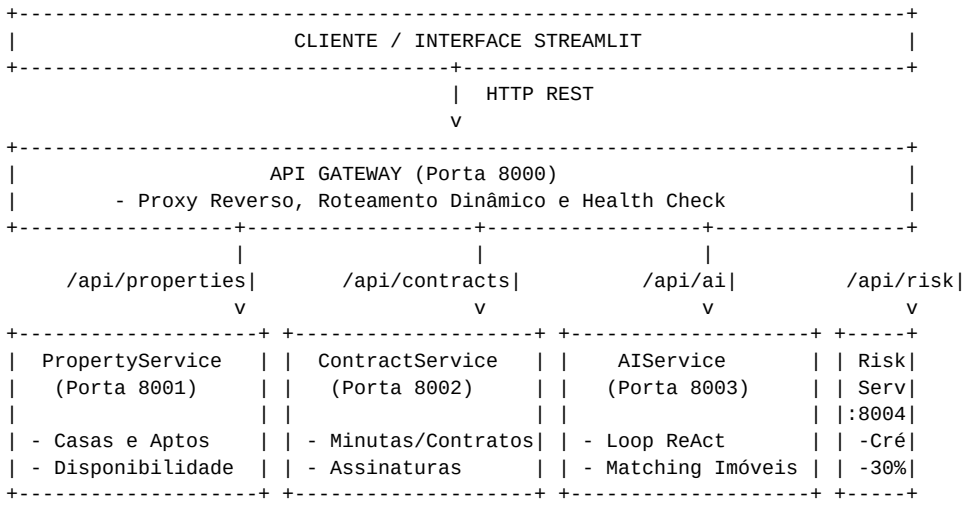



Figura 2: Arquitetura de Microsserviços com API Gateway e Portas de Rede Independentes

### 6.7 Especificação da API Gateway

O Listing 9 apresenta o código do API Gateway responsável por receber todas as requisições na porta 8000 e roteá-las dinamicamente para os serviços especialistas:

```
# Listing 9: API Gateway (Porta 8000)
import json
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.request

SERVICES = {
    "properties": "http://127.0.0.1:8001",
    "contracts": "http://127.0.0.1:8002",
    "ai": "http://127.0.0.1:8003",
    "risk": "http://127.0.0.1:8004"
}

class APIGatewayHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path in ["/health", "/"]:
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.end_headers()
            self.wfile.write(json.dumps({"gateway": "Online", "port": 8000, "services": SERVICES}).encode())
            return

        for prefix, target_url in SERVICES.items():
            if self.path.startswith(f"/api/{prefix}"):
                sub_path = self.path[len(f"/api/{prefix}"):] or "/"
                url = f"{target_url}{sub_path}"
                try:
                    with urllib.request.urlopen(url, timeout=3) as resp:
                        self.send_response(resp.status)
                        self.send_header("Content-Type", "application/json")
                        self.end_headers()
                        self.wfile.write(resp.read())
                    return
                except Exception as e:
                    self.send_response(502)
                    self.end_headers()
                    self.wfile.write(json.dumps({"error": str(e)}).encode())
                    return
```

### 6.8 Especificação dos Microsserviços

• **PropertyService (:8001):** Especialista no catálogo de imóveis (apartamentos e casas), especificações de quartos, vagas, aceitação de pets e status de disponibilidade.
• **ContractService (:8002):** Especialista no ciclo de vida de contratos, formalização de propostas e registro de assinaturas.
• **AIService (:8003):** Especialista no matching inteligente e motor consultivo via ReAct.
• **RiskService (:8004):** Especialista na avaliação de risco e análise de margem comprometida de renda.

### 6.9 Microsserviços em Execução e Portas Associadas

A Tabela 5 apresenta a relação dos 5 microsserviços em execução com suas portas dedicadas:

| Microsserviço | Porta | Protocolo | Status | Endpoint de Saúde (Health) |
|---|---|---|---|---|
| API Gateway | 8000 | HTTP/REST | ONLINE | http://127.0.0.1:8000/health |
| PropertyService | 8001 | HTTP/REST | ONLINE | http://127.0.0.1:8001/health |
| ContractService | 8002 | HTTP/REST | ONLINE | http://127.0.0.1:8002/health |
| AIService | 8003 | HTTP/REST | ONLINE | http://127.0.0.1:8003/health |
| RiskService | 8004 | HTTP/REST | ONLINE | http://127.0.0.1:8004/health |

## 7 Tecnologias Utilizadas

### 7.1 Framework e Aplicações

| Tecnologia | Versão | Uso no Projeto |
|---|---|---|
| Python | 3.11+ | Linguagem principal do framework e aplicações |
| abc (Abstract Base Classes) | Built-in | Implementação do Template Method e Hotspots |
| SQLite3 | Built-in | Persistência relacional no componente CSQLiteStorage |
| typing / dataclasses | Built-in | Tipagem estática e segurança na troca de dados |

### 7.2 Microsserviços

| Microsserviço | Porta | Linguagem | Framework | Banco de Dados |
|---|---|---|---|---|
| API Gateway | 8000 | Python | HTTP Server / Proxy | Sem banco (Stateless) |
| PropertyService | 8001 | Python | HTTP REST API | Catálogo em memória |
| ContractService | 8002 | Python | HTTP REST API | SQLite relacional |
| AIService | 8003 | Python | HTTP REST API | Motor ReAct |
| RiskService | 8004 | Python | HTTP REST API | Regras matemáticas |

### 7.3 Frontend e Infraestrutura

| Camada | Tecnologia / Ferramenta | Uso no Projeto |
|---|---|---|
| Frontend Web | Streamlit 1.58+ | Interface interativa para visualização dos 3 pilares |
| Diagramação | Mermaid.js / ASCII | Modelagem da arquitetura e do diagrama de classes |
| IA / Agente | Padrão ReAct | Ciclos de raciocínio Thought-Action-Observation |
| DevOps | Threading / Bash | Orquestração de portas e script automatizado run.sh |

## 8 Funcionamento do Sistema

### 8.1 Telas do Sistema e Funcionamento

A interface gráfica desenvolvida em Streamlit apresenta 6 abas interativas:

• **Aba 1 (Aplicações do Framework):** Exibe lado a lado os cards das 2 aplicações construídas, com os componentes vinculados aos hotspots e botão para disparo do Template Method canônico;

• **Aba 2 (Agente AI):** Formulário completo do inquilino (renda, pets, quartos, orçamento) com exibição transparente dos pensamentos e ações do agente ReAct;

• **Aba 3 (PropertyService):** Catálogo visual de casas e apartamentos cadastrados com detalhamento de custos calculados;

• **Aba 4 (ContractService):** Listagem de minutas geradas e botão para assinatura de contrato;

• **Aba 5 (Status dos Microsserviços):** Dashboard em tempo real verificando as portas 8000 a 8004;

• **Aba 6 (Conceitos):** Resumo dos conceitos de reuso e metodologia ágil.

## 8.2 Saída das Aplicações no Terminal

A execução direta das aplicações construídas a partir do framework gera a saída padronizada comprovando a associação dos hotspots:

```
$ python3 aluguel_residencial_app.py
============================================================
Aplicação 1 Inicializada: RentalFramework v1.0 - UFAL IC 2026
Componentes plugados nos Hotspots:
  - pricing_engine: CStandardPricingEngine (Tradicional 3x Caução)
  - risk_evaluator: CConservativeRiskEvaluator (Teto 30% Renda / Score 700+)
  - ai_agent: CReActAIAgent (Raciocínio ReAct com Análise Multicritério)
  - storage_strategy: CSQLiteStorage (SQLite em /tmp/aluguel_residencial.db)
============================================================

$ python3 aluguel_universitario_app.py
============================================================
Aplicação 2 Inicializada: RentalFramework v1.0 - UFAL IC 2026
Componentes plugados nos Hotspots:
  - pricing_engine: CSeasonalPricingEngine (Universitário/Flexível 1x Caução)
  - risk_evaluator: CFlexibleRiskEvaluator (Tolerância até 45% / Estudantil)
  - ai_agent: CRuleBasedAIAgent (Motor Heurístico de Menor Custo)
  - storage_strategy: CSQLiteStorage (SQLite em /tmp/aluguel_universitario.db)
============================================================
```

## 8.3 Ciclo de Raciocínio do Agente de IA (ReAct)

Ao consultar o agente inteligente, o sistema exibe o fluxo ReAct executado em tempo real:

```
[THOUGHT]     Agente analisando perfil de Samuel Silva: renda R$ 8.500, quartos>=2, pet=True.
[ACTION]      Executando busca no PropertyService (tipo=Qualquer, quartos_min=2, pets=True).
[OBSERVATION] 2 imóveis encontrados: APT-101 (R$ 2.903/mês) e CASA-301 (R$ 3.960/mês).
[THOUGHT]     Acionando RiskEvaluatorComponent para calcular comprometimento de renda.
[OBSERVATION] APT-101 consome 34.1% da renda (Risco Moderado); CASA-301 consome 46.5% (Risco Alto).
[FINAL ANSWER]Recomendado APT-101 no Centro com score de afinidade 79.4/100 e pré-aprovação.
```

## 9 GitHub

O código-fonte completo do projeto encontra-se estruturado e versionado:

• **Repositório Oficial:** https://github.com/samueeloliveiraa/framework-aluguel-rsma

A árvore do projeto organiza-se nas seguintes pastas:
• framework/ — Classe base abstrata, interfaces dos 4 hotspots, 8 componentes e 2 aplicações;
• microservices/ — 5 microsserviços autônomos (Portas 8000 a 8004) e orquestrador run_all.py;
• app.py — Interface gráfica em Streamlit;
• relatorio/ — Relatório técnico completo em Markdown e compilado em PDF;
• run.sh — Script executável com 1 comando para inicialização da interface.

## 10 Referências Bibliográficas

[1] GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading: Addison-Wesley, 1994.

[2] PREE, W. *Design Patterns for Object-Oriented Software Development*. Reading: Addison-Wesley, 1994.

[3] SOUZA, A. C. O.; NASCIMENTO, B. F. C.; SILVA, L. M. O.; JUNIOR, M. V. D. *Framework para Pagamentos na Web*. Universidade Federal de Alagoas (UFAL), 2025.

[4] DOMÍNGUEZ, A. H. *Notas de Aula da Disciplina de Reuso de Software e Metodologias Ágeis*. Instituto de Computação, Universidade Federal de Alagoas (UFAL), 2026.

[5] NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2. ed. Sebastopol: O'Reilly Media, 2021.

[6] YAO, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR), 2023.

[7] STREAMLIT DOCUMENTATION. *Build and share data apps*. Disponível em: <https://docs.streamlit.io>. Acesso em: nov. 2026.

[8] PYTHON SOFTWARE FOUNDATION. *abc — Abstract Base Classes*. Python 3 Documentation. Disponível em: <https://docs.python.org/3/library/abc.html>. Acesso em: nov. 2026.